

Link HTML, CSS and JS files

app.use()

function is used to mount the specified middleware function(s) at the path which is being specified. It is mostly used to set up middleware for your application.

Syntax:

```
app.use(path, callback)
```

Parameters:

- **path:** It is the path for which the middleware function is being called. It can be a string representing a path or path pattern or a regular expression pattern to match the paths.
- **callback:** It is a middleware function or a series/array of middleware functions.

Serving static files in Express

- ✓ To serve static files such as images, CSS files, and JavaScript files, use the `express.static` built-in middleware function in Express.

The function signature is:

```
express.static(root, [options])
```

The `root` argument specifies the root directory from which to serve static assets. For more information on the `options` argument, see `express.static`.

path.join()

- ✓ The `path.join()` method joins the specified path segments into one path.
- ✓ You can specify as many path segments as you like.
- ✓ The specified path segments must be strings, separated by comma.

__dirname

- ✓ `__dirname` is an environment variable that tells you the absolute path of the directory containing the currently executing file.

Folder structure

express ----1.js ----index.html ----1.css ----1.png Code: app.use (express.static (__dirname));	express ---1.js ----frontend ---- ---index.html ---- ---1.css ---- ---1.png Code: app.use (express.static ("frontend"));	express ----backend ----- --1.js ---index.html ---1.css ---1.png Here we need to join the path as we need to go to one step up to access the frontend files. Code: const sp = path.join (__dirname, "../"); app.use (express.static (sp));	express ----backend ----- --1.js ----frontend ----- ---index.html ----- ---1.css ----- ---1.png Here we need to join the path as we need to go to one step up to access the frontend files. Code: const sp = path.join (__dirname, "../frontend"); app.use (express.static(sp));
---	--	--	---

Load HTML file named index.html (All files in same folder)

express
|----1.js
|----index.html
|----1.css

Example

/express/index.html <html> <head><link rel="stylesheet" href="1.css"></head> <body><p class="lj_p"> File name must be index.html </p> </body> </html>	/express/1.css .lj_p { font-size:100; color:blueviolet; }	/express/1.js const express = require ("express") const app = express(); // name of file must be index.html app.use(express.static(__dirname)) app.listen (5200)
---	--	--

To run in terminal : go to “express” folder by command **cd express** and then **node 1.js**

Output in browser:

File name must be index.html

Load HTML file named index.html (create a frontend folder inside express folder and create index.html and 1.css inside it)

express

```
|---1.js
|----frontend
|-----|---index.html
|-----|---1.css
```

Example

/express/frontend/index.html <pre><html> <head> <link rel="stylesheet" href="1.css"> </head> <body> <p class="lj_p"> File name must be index.html </p> </body> </html></pre>	/express/frontend/1.css <pre>.lj_p { font-size:100; color:blueviolet; }</pre>	/express/1.js <pre>const express = require ("express") const app = express(); // name of file must be index.html app.use(express.static("frontend")) app.listen (5200)</pre>
--	---	---

To run in terminal : go to “express” folder by command **cd express** and then **node 1.js**

Output in browser:

File name must be index.html

Load HTML file named index.html (create a frontend folder inside express folder and create index.html and 1.css inside it. Also, create backend folder and create 1.js file inside it.)

express

```
|-----backend
|-----|---1.js
|-----frontend
|-----|---index.html
|-----|---1.css
```

Example

/express/frontend/index.html ml <html> <head> <link rel="stylesheet" href="1.css"> </head> <body> <p class="lj_p"> File name must be index.html </p> </body> </html>	/express/frontend/1.css s .lj_p { font-size:100; color:blueviolet; }	/express/backend/1.js const express = require ("express") const app = express(); const path = require ("path"); const staticpath = path.join (__dirname , "../frontend"); // name of file must be index.html app.use(express.static(staticpath)) app.listen (5200)
---	---	--

To run in terminal : go to "backend" folder by command **cd backend** and then **node 1.js**

Output in browser:

File name must be index.html

Load HTML file named index.html (create index.html and 1.css files inside express folder.
Create backend folder inside express folder and create 1.js file inside backend folder.)

express

```
|-----backend
|-----|--1.js
|---index.html
|---1.css
```

Example

/express/index.html <html> <head><link rel="stylesheet" href="1.css"></head> <body><p class="lj_p"> File name must be index.html </p> </body> </html>	/express/1.css .lj_p { font-size:100; color:blueviolet; }	/express/backend/1.js const express = require ("express") const app = express(); const path = require ("path"); const staticpath = path.join (__dirname, "../"); // name of file must be index.html app.use(express.static(staticpath)) app.listen (5200)
---	--	--

To run in terminal : go to "backend" folder by command **cd backend** and then **node 1.js**

Output in browser:

File name must be index.html

load HTML file using `res.sendFile()`

express/frontend/1.html <html> <head> <link rel="stylesheet" href="1.css"> </head> <body> <p class="lj_p"> Example using sendFile method.</p> </body> </html>	express/frontend/1.css .lj_p { font-size:100; color:blueviolet; }	express /backend/1.js const express = require ("express") const app = express(); const path = require ("path"); const staticpath = path.join (__dirname , "../frontend"); //to load css file which is linked in html file app.use(express.static(staticpath)) //Another way to load HTML file app.get('/',(req,res)=>{ res.sendFile(staticpath + "/1.html") ; }) app.listen (5200)
---	--	--

Load HTML file named other than index.html file.

express/frontend/2.html <html> <head> <link rel="stylesheet" href="2.css"></head> <body> <p class="lj_p">File name is other than index.html.</p> </body> </html>	express/frontend/2.css .lj_p { font-size:100; color:blueviolet; }	express/backend/2.js const express = require ("express") const path = require ("path"); const app = express(); const staticpath = path.join (__dirname , "../frontend"); app.use (express.static(staticpath ,{ index:"2.html" })); app.listen(5200)
--	--	---

Example to understand concept of all files in different folders.

Express

```
|-----js
|-----|---1.js
|-----html
|-----|---1.html
|-----css
|-----|---1.css
|-----image
|-----|--- folderstructure.png
```

express/js/1.js

```
const express = require("express");
const app = express();
const path = require("path");

var css_path = path.join(__dirname, "../css")
var img_path = path.join(__dirname, "../image")
var html_path = path.join(__dirname, "../html")

app.use(express.static(css_path))
app.use(express.static(img_path))
app.use(express.static(html_path, {index: "1.html"}))

app.listen(5001, () => console.log("Started"))
```

express/html/1.html

```
<html>
  <head>
<link rel="stylesheet" href="1.css">
  </head>
  <body>
    <p class="lj_p"> Hello </p>
    
```

```
</body>  
</html>
```

express/css/1.css

```
.lj_p{  
  font-size: 60px;  
  color:purple;  
  text-align: center;  
}  
img{ width:400px; height:auto;}
```

Note that if you don't want to use `path module` to join the path then you can use below code.

```
app.use(express.static("../frontend"));
```

OR

```
app.use(express.static(__dirname + "../frontend"));
```

Problem without path

- May break on different systems
- Path may not resolve correctly
- Not reliable for real projects

Handling Forms & Form Data

- ✓ Handling forms and form data means sending user input from the browser to the server. This is done using HTTP methods, mainly GET and POST. When a user fills a form and submits it, the browser sends a request to the server.
- ✓ The GET method is used when the data needs to be retrieved from the server. In this method, the form data is sent through the URL as a query string. The data appears after the question mark in the URL. Because the data is visible in the URL, it is not secure. The amount of data that can be sent using GET is limited. In Express, the data sent using GET is accessed using `req.query`. GET is commonly used for search operations, filters, and fetching data where security is not important.
- ✓ The POST method is used when the data needs to be sent to the server for processing or storage. In this method, the data is sent inside the request body and is not visible in the URL. This makes POST more secure compared to GET. POST can handle large amounts of data. In Express, the data sent using POST is accessed using `req.body`, but it requires middleware such as `express.json` or `express.urlencoded` to parse the data. POST is commonly used for form submissions like login, registration, and uploading data.
- ✓ The main difference between GET and POST is how and where the data is sent. GET sends data through the URL, while POST sends data through the request body. GET is faster and simple but less secure, whereas POST is more secure and suitable for sensitive or large data.

GET and POST

- ✓ GET and POST both are two common HTTP requests used for building REST API's.
- ✓ GET requests are used to send only limited amount of data because data is sent into header while POST requests are used to send large amount of data because data is sent in the body.
- ✓ Express.js facilitates you to handle GET and POST requests using the instance of express.

Get method

It facilitates you to send only limited amount of data because data is sent in the header. It is not secure because data is visible in URL bar.

Example 1

Write express script to get form data using get method and display data in JSON format in next page named “process_get”

index.html

```
<html>
```

```
<body>
<form action="/process_get" method="get">
First Name: <input type="text" name="first_name"> <br>
Last Name: <input type="text" name="last_name">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

example1.js

```
var express = require('express');
var app = express();
app.use(express.static(__dirname))
app.get('/process_get', function (req, res) {
var fn = req.query.first_name;
var ln = req.query.last_name
  console.log(req.query); //req.query is used to get the submitted data in json object format.
  To access particular key value we have to write "req.query.first_name"
  res.send(`Welcome ${fn} ${ln}`);
})
app.listen(8000)
```

Example 2

Write express js script to print message in next line splitting by "." And use get method to submit the data. HTML file contains form of text area for the message and submit button.

2.html

```
<html>
<head><title>Get Method</title></head>
<body>
  <form method="get" action="/login">
    <textarea rows="10" cols="10" name="message">Hi.Hello.how are you
  </textarea>
  <button name="Submit" value="Submit">Submit</button>
  </form>
</body>
</html>
```

2.js

```
var express = require("express");
var app = express();

app.use(express.static(__dirname,{index: '2.html'}));

app.get("/login",(req,res)=>{
  res.set("content-type","text/html");

  t1 = (req.query.message).split(".");
  for(i in t1){
    res.write(t1[i]+ "</br>");
  }
  res.send();
});
app.listen(5122);
```

Output:

Hi
Hello
how are you

Example 3

Write express js script to perform tasks as asked below.

1. Create one HTML file which contains two number type input fields, one dropdown which contains options like (select, addition, subtraction, multiplication, division) and one submit button.
2. The input fields must contain the value greater than 0 else it will give a message "Please enter the valid number". Also, user must select any of the formula from the dropdown else give a message "You have not selected any formula". (Message will be displayed on "/calc" page.)
3. If one formula is selected and numbers are entered then respective calculations will be performed on the page "/calc".
4. Use get method to request data.

calc.js

```
var express = require("express");
var app = express();
```

```
app.use(express.static(__dirname,{index:'calc.html'}));
```

```
app.get("/calc",(req,res)=>{
  res.set("content-type","text/html");
  var n1 = parseInt(req.query.n1);
  var n2 = parseInt(req.query.n2);
  if ((n1>0) && (n2>0)) {
    if(req.query.formula == "addition"){
      a = n1+n2;
      res.write("<h1>Addition is : " + a + "</h1>");
    }
    else if(req.query.formula == "subtraction"){
      s = n1-n2;
      res.write("<h1>Subtraction is : " + s + "</h1>");
    }
    else if(req.query.formula == "multi"){
      m = n1*n2;
      res.write("<h1>Multiplication is : " + m + "</h1>");
    }
    else if(req.query.formula == "div"){
      d = n1/n2;
      res.write("<h1>Division is : " + d + "</h1>");
    }
    else{
      res.write("<h1>You have not selected any formula.</h1>");
    }
  }else{
    res.write("<h1>Please enter the valid number/s.</h1>");
  }
  res.send()
})
app.listen(6012);
```

calc.html

```
<html>
<body>
  <form action="/calc" method="get">
    <fieldset>
      <legend>Calulator:</legend>
      <input type="number" name="n1"><br><br>
```

```
<input type="number" name="n2"><br><br>
```

```
<select name="formula" >
```

```
  <option value="">Select formula</option>
```

```
  <option value="addition">Addition</option>
```

```
  <option value="subtraction">Subtraction</option>
```

```
  <option value="multi">Multiplication</option>
```

```
  <option value="div">Division</option>
```

```
</select><br><br>
```

```
<input type="submit" value="Submit">
```

```
</fieldset>
```

```
</form>
```

```
</body>
```

```
</html>
```

Output:

← → ↻ ⓘ localhost:6012/calc?n1=1&n2=7&formula=addition

Addition is : 8

Post method

It facilitates you to send large amount of data because data is sent in the body. Post method is secure because data is not visible in URL.

Please note that new version of express contains built-in body-parser module. We don't need to install it and import it to our program.

We can access it's functionality by just adding below line to our program.

```
var express = require("express");  
app.use(express.urlencoded({ extended: false }));
```

Reference

If above won't work properly then follow the following steps of installation and importing module to the program.

Installation

npm install body-parser

API

```
var bodyParser = require('body-parser')  
app.use(bodyParser.urlencoded({ extended: false }));
```

- Body-parser parses is an HTTP request body that usually helps when you need to know more than just the URL being hit.
- It provides four express middleware for parsing JSON, Text, URL-encoded, and raw data sets over an HTTP request body.
- Using **body-parser** allows you to access **req.body** from within routes and use that data.

- The **urlencoded()** function is a built-in middleware function in Express.
- It parses incoming requests with URL-encoded payloads and is based on a body parser.
- **express.urlencoded()** expects request data to be sent encoded in the URL, usually in strings or arrays.
- **extended** : The extended option allows to choose between parsing the URL-encoded data with the **querystring** library (**when false**) or the **qs** library (**when true**).

Specify the extended option as either true or false. Here's what that means:

- extended: false → Uses the **querystring** library (doesn't support rich objects).
- extended: true → Uses the **qs** library (allows rich objects and arrays).

Example 1

Write express script to retrieve form data using post method and display data in JSON format in next page named “process”

p.html

```
<html>
<body>
<form action="/process" method="post">
First Name: <input type="text" name="first_name"> <br>
Last Name: <input type="text" name="last_name">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

p.js

```
var express = require('express');
var app = express();

app.use(express.urlencoded({ extended: false }));

app.use(express.static(__dirname,{index: 'p.html'}));

app.post('/process', function (req, res) {
  console.log(req.body.first_name+" "+req.body.last_name)
  res.send(req.body);
})

app.listen(7002)
```

Example 2

Build an Express.js app with:

1. An HTML form with username, password, and submit button. Use POST method.
2. On visiting /, show the form.
3. On submitting to /login:
 - If username is "admin", display: "Welcome admin"
 - Else, show: "Please login with admin name" and a link back to the form.

po2.js

```
var express = require("express");
```

```
var app = express();
app.use(express.urlencoded());
app.use(express.static(__dirname,{index: 'po2.html'}));
app.post("/login", (req, res) => {
  if (req.body.uname == 'admin') {
    res.write("<h1> Welcome " + req.body.uname + "</h1>")
  }
  else {
    res.write("<h1 style='color:red'>Please login with admin name</h1> <a href='/'>Try again</a>");
  }
  res.send();
})
app.listen(5222);
```

po2.html

```
<form method="post" action="/login">
<div class="lj-div">
  <label>User name:</label>
  <input type="text" name="uname" ></input>
</div>
<div class="lj-div">
  <label>Password:</label>
  <input type="text" name="password" ></input>
</div>
<button name="Submit" value="Submit">Submit</button>
</form>
```


Cookie

How cookies work

- When a user visits a cookie-enabled website for the first time, the browser will prompt the user that the web page uses cookies and request the user to accept cookies to be saved on their computer. Typically, when a user makes a request, the server responds by sending back a cookie (among many other things).
- This cookie is going to be stored in the user's browser. When a user visits the website or sends another request, that request will be sent back together with the cookies. The cookie will have certain information about the user that the server can use to make decisions on any other subsequent requests.

For example, **Facebook** from a browser.

When you want to access your Facebook account, you have to log in with the correct credentials. When you first make a login request and the server verifies your credentials, the server will send your Facebook account content. It will also send cookies to your browser. The cookies are then stored on your computer and submitted to the server with every request you make to that website. A cookie will be saved with an identifier that is unique to that user.

When you revisit Facebook, the request you make, the saved cookie, and the server will keep track of your login session and remember who you are and thus keep you logged in.

The main difference between a session and a cookie

The major difference between [sessions](#) and [cookies](#) is that sessions live on the server-side (the webserver), and cookies live on the client-side (the user browser). Sessions have sensitive information such as usernames and passwords. This is why they are stored on the server. Sessions can be used to identify and validate which user is making a request.

As we have explained, cookies are stored in the browser, and no sensitive information can be stored in them. They are typically used to save a user's preferences.

- [cookie-parser](#) - cookie-parser looks at the headers in between the client and the server transactions, reads these headers, parses out the cookies being sent, and saves them in a browser. In other words, cookie-parser will help us create and manage cookies depending on the request a user makes to the server.

Run the following command to install these NPM packages:

```
npm install cookie-parser
```

We will create a simple example to demonstrate how cookies work.

Step 1 - Import the installed packages

To set up a server and save cookies, import the cookie parser and express modules to your project. This will make the necessary functions and objects accessible.

```
const express = require('express')
const cookieParser = require('cookie-parser')
```

Step - 2 Get your application to use the packages

You need to use the above modules as middleware inside your application, as shown below.

```
//setup express app
const app = express()
// let's you use the cookieParser in your application
app.use(cookieParser());
```

This will make your application use the cookie parser and Express modules.

Example

app.js

```
var express = require('express');
var app = express();
var cp = require('cookie-parser');
app.use(cp());

app.get('/cookie', function(req, res){
  res.cookie('name', 'Express JS');
  res.cookie('fname', 'express'); //Sets fname = express
  res.cookie('lname', 'js'); //Sets lname = js
  res.cookie('ID', '2', { expires: new Date(Date.now()+10000)}); //expires after 10 secs
  res.cookie('email', 'express@gmail.com', {maxAge:2000}); //expires after 2 secs

  res.clearCookie('fname');

  //show the saved cookies
  res.send(req.cookies); //Use req.cookies method to check the saved cookies
});
app.listen(3000);
```

When the above route is executed from a browser, the client sends a get request to the server. But in this case, the server will respond with a cookie and save it in the browser.

Open <http://localhost:3000/cookie> your browser and access the route.

To confirm that the cookie was saved, go to your browser's [Inspect element > select the application tab > cookies > select your domain URL.](#)

localhost:3000/cookie

```

{
  "name": "Express JS",
  "lname": "js",
  "ID": "2",
  "email": "express@gmail.com"
}

```

Application

Storage

Local storage

Session storage

IndexedDB

Cookies

http://localhost:3000

Name	Value	Domain	Path	Expires / Max-...	Size
ID	2	localhost	/	2024-06-12T1...	3
email	express%40gmail.com	localhost	/	2024-06-12T1...	24
lname	js	localhost	/	Session	7
name	Express%20JS	localhost	/	/ session	16

After 10 seconds check cookies

Filter

Name	Value	Domain	Path	Expires / Max-...	Size
lname	js	localhost	/	Session	7
name	Express%20JS	localhost	/	Session	16

To check if your cookie is set or not, just go to your browser, fire up the console, and enter – **console.log(document.cookie);**

Example

Write an express js script to set cookies of submitted values of form. Perform following tasks.

- Create a HTML file which contains a form with fields first name, last name, password and a submit button.
- Once form submitted, store all these entered values to the respective cookies on '/next' page.
- Then redirect user to "/admin" page and clear the cookie set for the last name. Display remaining set cookie values on this page. (Using post method)

c1.html

```

<html>
  <body>

```

```
<form method="post" action="/next">
  First Name : <input type = "text" name="fname" ></input>
  Last Name : <input type = "text" name="lname" ></input>
  Password : <input type = "password" name="password" ></input>
  <button name="Submit" value="Submit">Submit</button>
</form>
</body>
</html>
```

c1.js

```
var express = require("express");
var app = express();
var cp = require("cookie-parser");
app.use(cp());
app.use(express.static(__dirname,{index: 'c1.html'}));
app.use(express.urlencoded())
app.post("/next",(req,res,next)=>{
  res.cookie("fname", req.body.fname);
  res.cookie("password", req.body.password);
  res.cookie("lname", req.body.lname);
  res.redirect("/admin");
})
app.get("/admin",(req,res)=>{
  res.clearCookie('lname');
  res.write(" Welcome : " + req.cookies.fname);
  res.write(" Lname : " + req.cookies.lname);
  res.write(" Password : " + req.cookies.password);

  res.send()
});
app.listen(3000);
```

Welcome : abc Lname : undefined Password : 123



We have cleared lname cookie. So, req.cookies.lname is undefined.

Example

You have been assigned to develop a user feedback form for a website using Express.js and cookies.

Implement the following requirements:

- Create a form with the following fields:
 - Name (input field)
 - Email (input field)
 - Message (textarea field)
 - Rating (radio buttons: Bad, Average, Good, Very Good, Excellent)
- When the user submits the form, store their feedback information (name, email, message, and rating) in a cookie named "feedback" that expires in 10 seconds.
- Display a confirmation message to the user after successfully submitting the form & Create a link to display the feedback details stored in the "feedback" cookie.
- When the user clicks to the link, retrieve the feedback information from the cookie and display it on the page also include a link on the feedback details page to Logout. When the user clicks the link, user redirected to home page.
- After, 10 seconds it will give message "No Feedback available" message to user

feedback.js

```
const express = require('express');
const cp = require('cookie-parser');
const app = express();
app.use(cp());
app.use(express.urlencoded());
app.use(express.static(__dirname, {index: 'feedback.html'}))

app.post('/submit-feedback', (req, res) => {
  const { name, email, message, rating } = req.body;
  // Create the feedback object
  const feedback = {name, email, message, rating};
  // Set the feedback cookie with a 10-second expiration
  res.cookie('feedback', feedback, { maxAge: 10000 });
  res.send('Thank you for your feedback! <br> <a href="/feedback-details"> Show Feedback </a>');
});

app.get('/feedback-details', (req, res) => {
  const feedback = req.cookies.feedback;
```

```

if (feedback) {
  res.send(`
    <h1>Feedback Details</h1>
    <p><strong>Name:</strong> ${feedback.name}</p>
    <p><strong>Email:</strong> ${feedback.email}</p>
    <p><strong>Message:</strong> ${feedback.message}</p>
    <p><strong>Rating:</strong> ${feedback.rating}</p>
    <a href="/" > logout </a>`);
} else {
  res.send('No feedback available.');
```

```

}
```

```

});
```

```

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

feedback.html

```

<body>
  <h1>User Feedback Form</h1>
  <form action="/submit-feedback" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required><br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required><br><br>

    <label for="message">Message:</label>
    <textarea id="message" name="message" required></textarea><br><br>

    <label for="rating">Rating:</label>
    <input type="radio" id="rating1" name="rating" value="Bad" required>
    <label for="rating1">Bad</label>
    <input type="radio" id="rating2" name="rating" value="Average" required>
    <label for="rating2">Average</label>
    <input type="radio" id="rating3" name="rating" value="Good" required>
    <label for="rating3">Good</label>
    <input type="radio" id="rating4" name="rating" value="Very Good" required>
    <label for="rating4">Very Good</label>
    <input type="radio" id="rating5" name="rating" value="Excellent" required>
```

```
<label for="rating5">Excellent</label><br><br>
<input type="submit" value="Submit">
</form>
</body>
```

Session

HTTP is stateless; in order to associate a request to any other request, you need a way to store user data between HTTP requests. Cookies and URL parameters are both suitable ways to transport data between the client and the server. But they are both readable and on the client side. Sessions solve exactly this problem. You assign the client an ID and it makes all further requests using that ID. Information associated with the client is stored on the server linked to this ID. Install the using the command:

```
npm install express-session
```

To set up the session, you need to set a couple of Express-session options, as shown below.

```
const oneDay = 1000 * 60 * 60 * 24; // creating 24 hours from milliseconds
app.use(sessions({
  secret: "thisismysecretkey",
  saveUninitialized:true,
  cookie: { maxAge: oneDay },
  resave: false
}));
```

secret

a random unique string key used to authenticate a session. It is stored in an environment variable and can't be exposed to the public. The key is usually long and randomly generated in a production environment.

resave

- **true** → Saves session on **every request** (even if not modified).
- **false** → Saves session **only if modified** (recommended). Setting false **reduces unnecessary session writes** and prevents race conditions.

saveUninitialized

- **true** → Saves **new but unmodified** sessions.
- **false** → Saves a session **only if modified** (recommended for login-based apps). Reducing **unnecessary session storage**. Prevents **storing empty sessions** for visitors who don't log in.

cookie

{ maxAge: oneDay } - this sets the cookie expiry time. The browser will delete the cookie after the set duration elapses. The cookie will not be attached to any of the requests in the future. In this case, we've set the maxAge to a single day as computed by the following arithmetic.

Examples

Write express js script using session to display how many times a user visited a website. If user is visiting a website for the first time then display "Welcome! Thank you for visiting our website!" else display the count How many times he/she has visited the website for that particular session.

```
const express=require("express");
const app=express();
const sess=require("express-session");
app.use(sess(
  {
    resave:true,
    saveUninitialized:true,
    secret:"LJU123"
  }
));
app.get("/",(req,res)=>
{
  if(req.session.page_views)
  {
    req.session.page_views++;
    res.send(`<h1 style="color:blue;"> You have visited page ${req.session.page_views} times
<h1>`);
  }
  else{
    req.session.page_views=1;
    res.send(`<h1 style="color:green;"> Welcome! Thank you for visiting our
website!<h1>`);
  }
});
app.listen(8003,()=>{console.log("server running at 8003");});
```

Example

write a script to meet following requirements:

- Create index.html file page which contains form(username,password,login button). and open it on localhost.
- After clicking submit button, it should jump on “savesession” page. Store username and password in session.
- After saving session, redirect to “fetchsession” page and read value. Put a LOGOUT link here.
- Jump on delete session page after clicking LOGOUT link.
- **Destroy the session on this page and redirect to index.html page.**

session.js

```
var express = require("express");
var app = express();
var es = require("express-session");
app.use(es({
  resave:false,
  saveUninitialized:false,
  secret:"lju1"
}));
app.use(express.static(__dirname));

app.get("/savesession",(req,res)=>{
  req.session.uname = req.query.uname;
  req.session.password = req.query.password;
  res.redirect("fetchsession")
})
app.get("/fetchsession",(req,res)=>{
  res.write("<h1>Welcome " + req.session.uname + "</h1>")
  res.write("<a href='/deletesession'>Logout</a>")
  res.send();
});
app.get("/deletesession",(req,res)=>{
  req.session.destroy()
  res.redirect('/')
});
app.listen(6177);
```

index.html

```
<html>
  <body>
```

```

<center>
<form method="get" action="/saveession">
  <div><input type = "text" name="uname" placeholder="Username" ></input></div>
  <div><input type = "password" name="pass" placeholder="Password"></input></div>
  <input class="lj_in" type="submit" name="submit" value="Login">
</form>
</center>
</body>
</html>

```

Example

write a script to meet following requirements:

- Create session.html file page which contains form(username,password,login button). and open it on localhost.
- After clicking submit button, it should jump on “**save**” page. Store username and password in session.
- After saving session, redirect to “**fetchdata**” page and read value. On this page check **authentication** of user. **User name and password must be “admin” and “admin@123” respectively.**
 - If this condition is true then display welcome admin and display logout link on this page(fetchdata).
 - By clicking on logout link user should jump to “**destroy**” page and destroy the session there and display the message “Session destroyed”. And give the link of “login” under that message. By clicking that link user will be redirected to the home page.
 - Else display “Please enter valid username and password” and login link on this page(fetchdata).

session.html

```

<body>
<center>
<form method="get" action="/save">
  <div><input type = "text" name="uname" placeholder="Username" ></input></div>
  <div><input type = "password" name="pass" placeholder="Password"></input></div>
  <input type="submit" name="submit" value="Login">
</form>
</center>
</body>

```

Session.js

```
var express = require("express");
```

```
var app = express();
var es = require("express-session");
app.use(es({
  resave:false,
  saveUninitialized:false,
  secret:"lju1"
}));
app.use(express.static(__dirname,{index:'session.html'}));

app.get("/save",(req,res)=>{
  req.session.username = req.query.username;
  req.session.password = req.query.pass;
  res.redirect("/fetchdata")
})
app.get("/fetchdata",(req,res)=>{
  if(req.session.username == "admin" && req.session.password=="admin@123"){
    res.write("<h1 style='color:green;'>Welcome Admin</h1>")
    res.write("<a href='/destroy'>Logout</a>")
  }else{
    res.write("<h1 style='color:red;'>Please enter valid username or password</h1><a href='/'>Login</a>")
  }
  res.send();
});
app.get("/destroy",(req,res)=>{
  req.session.destroy()
  res.write("<h1>Session Destroyed</h1><a href='/'>Login</a>")
  res.send();
});
app.listen(6177);
```

Middleware Examples with get/post methods

Example 1

Write express js script to perform following tasks.

1. Create one html file which contains one text field for name, email field and checkbox for subscription. Html file will be loaded on home page. Email and name fields are required fields.
2. On login page welcome user and email id data should be printed.
 - a. If user checked the subscription then "Thank you for the subscription" message will be printed and "logout" link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.
 - b. If user has not opted for the subscription then "You can subscribe to get daily updates" message will be printed and "subscribe" link will be displayed under the message.
 - c. If user clicks subscribe link then he/she will be redirected to the subscription page. In this page "Thank you for the subscription" message will be printed and "logout" link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.

Use concept of the middleware and you can use any of http methods(get/post).

t1.js

```
var express = require("express");
var app = express();

app.use(express.static(__dirname,{index:'t1.html'}));

app.use(express.urlencoded());

app.post("/login",(req,res,next)=>{
  res.set("content-type","text/html");
  res.write("<center><h1>Welcome " + req.body.name + "</h1>");
  res.write("<center><h2>Your email id is " + req.body.email + "</h2>");
  next();
});

app.post("/login",(req,res)=>{
  if(req.body.newsletter == "on"){
    res.write("<h3>Thank you for your subscription</h3><a href='/'>Logout</a>");
  }else{
```

```

    res.write("<h3>You can subscribe to get daily updates</h3><a
href='/subscribe'>Subscribe</a></center>");
  }
  res.send();
});
app.get("/subscribe",(req,res)=>{
  res.set("content-type","text/html");
  res.write("<h3>Thank you for your subscription</h3></center><a href='/'>Logout</a>");
  res.send();
});
app.listen(5001);

```

t1.html

```

<form action="/login" method="post">
  <label for="name">Name:</label>
  <input type="text" name="name" required><br><br>

  <label for="email">Email:</label>
  <input type="email" name="email" required><br><br>
  <input type="checkbox" name="newsletter" id="newsletter">
  <label for="newsletter">Subscribe?</label><br><br>
  <input type="submit" value="Submit">
</form>

```

Output:

The screenshot shows a web browser at localhost:5001. The top part displays a login form with fields for 'Name' and 'Email', a 'Subscribe?' checkbox, and a 'Submit' button. Below the form, there are two side-by-side panels. The left panel, titled 'Welcome lju', shows 'Your email id is lju@gmail.com' and 'Thank you for your subscription', with a 'Logout' link. The right panel, also titled 'Welcome lju', shows 'Your email id is lju@gmail.com' and 'You can subscribe to get daily updates', with a 'Subscribe' link.

Example 2

Write an express.js script to load an HTML file having username and password and submit button. On clicking submit button. It should jump on "check" page using "POST"

method. If username is "admin" , then jump on next middleware to print "welcome... admin" , if username is not "admin" , then stay on same middleware to print "warning msg" in red color.

Index.html

```
<html>
<body>
<form action ="/check" method="post">
Username: <input type="text" name="uname"/>
password: <input type="text" name="pwd"/>
<input type="submit"/>
</form>
</body>
</html>
```

App.js

```
const express=require("express");
const app=express();
app.use(express.urlencoded({extended:false}));
app.use(express.static(__dirname));
app.post("/check",(req,res,next)=>{
if(req.body.uname=="admin")
{
  next();
}
else {
  res.send("<h1 style='color:red'>wrong credentials</h1>") }
});
app.post("/check",(req,res)=>
{
  res.write("<h1>welcome..." + req.body.uname+ "</h1>")
  res.send();
}
).listen(3001);
```

RESTful API

- ✓ [REST](#), which stands for **REpresentational State Transfer**, is a software development architecture that defines a set of rules for communication between a client and a server. Let's break this down a little more:
- ✓ A **REST client** is a code or app used to communicate with REST servers.
- ✓ A **server** contains resources that the client wants to access or change.
- ✓ A **resource** is any information that the API can return.
- ✓ A REST API, also known as a RESTful API, is an API that conforms to the REST architecture. These APIs use the [HTTP](#) protocol to access and manipulate data on the server.
- ✓ An popular architectural style of how to structure and name APIs and the endpoints is called **REST(Representational State Tansfer)**. An API is always needed to create mobile applications, single page applications, use AJAX calls and provide data to clients.
- ✓ We will be using JSON as our transport data format as it is easy to work. we are using Router from Express, and we are exporting it using module.exports. So, our app will work fine.
 - ✓ Use the express.Router class to create modular, mountable route handlers. A Router instance is a complete middleware and routing system; for this reason, it is often referred to as a "mini-app".
 - ✓ The following example creates a router as a module, loads a middleware function in it, defines some routes, **and mounts the router module on a path in the main app.**

Example

Write an expressJS code in which RESTapi is created for json object named data which contains name,id,branch,city and contact properties.

- ✓ On `"/api"` page it should display all the content.
- ✓ Upon passing id on the browser url it should display the content having that id. (i.e. on `localhost:7899/api/101`)
- ✓ Upon passing branch on the browser url it should display the content having that branch. (i.e. on `localhost:7899/api/cse`)

api.js

```
const expr = require("express");
const router = expr.Router();
const data=[
  { id:101,name:"ABC",branch:"CSE",contact:9876543210,city:"Ahmedabad" },
  { id:102,name:"BCD",branch:"CE",contact:9876543210,city:"Ahmedabad" },
  { id:103,name:"XYZ",branch:"CSE",contact:9876543210,city:"Rajkot" },
  { id:104,name:"PQR",branch:"IT",contact:9876543210,city:"Ahmedabad" },
  { id:105,name:"ABC",branch:"CSE",contact:9876543210,city:"Surat" },
  { id:106,name:"ABC",branch:"IT",contact:9876543210,city:"Rajkot" }
]
```



```
router.get("/",(req,res)=>{
  res.set("content-type","text/html")
  for(i of data){
    res.write("<h3>ID: " + JSON.stringify(i.id) + ", Name: " + i.name + ", Branch: " + i.branch + ",
Contact: " + i.contact + ", City: " + i.city + "</h3>");
  }
  res.send();
})
```

```
router.get("/:id",(req,res)=>{
  var current_data = data.filter((i1)=> i1.id == req.params.id)
  if(current_data.length > 0){
    res.send(current_data);
  }else{
    res.send("Not Found")
  }
})
```

```
router.get("/branch/:branch",(req,res)=>{

  var cd = data.filter((b)=>b.branch.toLowerCase() == req.params.branch.toLowerCase())
  if(cd.length > 0){
    res.send(cd);
  }else{
    res.send("Not Found")
  }
})
module.exports = router;
```

main.js

```
const expr = require("express");
const app = expr();
const api = require("./api")
app.use("/api",api);
app.listen(7899);
```

To run:
node main.js

Check output in browser:

<http://localhost:7899/api>

ID: 101, Name: ABC, Branch: CSE, Contact: 9876543210, City: Ahmedabad

ID: 102, Name: BCD, Branch: CE, Contact: 9876543210, City: Ahmedabad

ID: 103, Name: XYZ, Branch: CSE, Contact: 9876543210, City: Rajkot

ID: 104, Name: PQR, Branch: IT, Contact: 9876543210, City: Ahmedabad

ID: 105, Name: ABC, Branch: CSE, Contact: 9876543210, City: Surat

ID: 106, Name: ABC, Branch: IT, Contact: 9876543210, City: Rajkot

<http://localhost:7899/api/101>

`[{"id":101,"name":"ABC","branch":"CSE","contact":9876543210,"city":"Ahmedabad"}]`

<http://localhost:7899/api/branch/it>

`[{"id":104,"name":"PQR","branch":"IT","contact":9876543210,"city":"Ahmedabad"}, {"id":106,"name":"ABC","branch":"IT","contact":9876543210,"city":"Rajkot"}]`